

Building Your Academic Website with RStudio & Quarto

Your Friendly Neighborhood, Spider-Rony

July 10, 2025

Contents

1	Why Bother with an Academic Website?	2
2	What Are the Options Out There?	2
3	Requirements: What You'll Need	2
3.1	Part 1: Install R, the Engine	2
3.1.1	For macOS	2
3.1.2	For Windows	3
3.2	Part 2: Install RStudio, the Cockpit	3
3.3	Part 3: Get Git and a GitHub Account	3
3.3.1	Install Git	3
3.3.2	Create a GitHub Account and Install GitHub Desktop	3
3.4	Final Check	3
4	Let's Build! Creating Your Quarto Website	4
4.1	Step 1: Create a New Project	4
4.2	Step 2: The Key Files - Examples & Explanation	5
4.2.1	The Brain: <code>_quarto.yml</code>	5
4.2.2	The Face: <code>index.qmd</code>	6
4.3	Step 3: Render and Preview	6
5	Hosting on GitHub Pages	6
5.1	Step 1: Connect Your Project with GitHub Desktop	6
5.2	Step 1.5: Configure GitHub Pages	7
5.3	Step 2: Push and Update Your Website to GitHub	7
6	Troubleshooting: Common Issues with GitHub Pages, Jekyll, and Quarto	7
6.1	Website Not Updating or Not Showing Up	7
6.2	Jekyll Processing Issues (<code>.nojekyll</code> File)	8
6.3	Broken Links or Missing Images	8
6.4	Changes Not Appearing	8
6.5	404 Errors on Subpages	8
6.6	YAML Formatting Errors	8
6.7	Quarto Version Issues	9
6.8	More Help	9

My bad, I had no time to include images, but we will go through the process step-by-step, so you can follow along easily.

Abstract

This tutorial is your one-hour guide to creating a professional, easy-to-maintain academic website. We'll be using RStudio and Quarto. By the end of this session, you'll have (*should have*) a functional website ready to be shared with the world via GitHub. No prior web development experience needed!

1 Why Bother with an Academic Website?

Before we dive in, let's quickly talk about why you, as a PhD student, should have a personal website. It's your corner of the internet to:

- **Showcase your work:** Share your publications, presentations, and ongoing projects.
- **Control your narrative:** Present yourself and your research exactly how you want.
- **Network:** Make it easy for potential collaborators, conference organizers, and future employers to find you and your work.
- **Share resources:** Post teaching materials, code, datasets, or a blog.

2 What Are the Options Out There?

Quarto is great, but it's not the only way to build your website. If this becomes too convoluted, here are a few other popular choices for academics:

Google Sites:

Pros: Incredibly easy to use, drag-and-drop interface, free.

Cons: Limited customization, looks a bit "Googley," not ideal for showcasing code or data-heavy projects.

WordPress:

Pros: Extremely customizable with themes and plugins. Great for blogging.

Cons: Can have a steeper learning curve, requires hosting (which can cost money), and needs regular maintenance (updates, security).

GitHub Pages with Jekyll/Hugo:

Pros: Highly customizable, great for technical people, free hosting on GitHub.

Cons: Requires more comfort with the command line, Git, and Markdown/HTML. It's what Quarto uses under the hood, but Quarto makes it much easier!

So why Quarto? It hits the sweet spot: it's powerful and customizable like Jekyll/Hugo but with a much friendlier workflow, especially if you're already using RStudio.

3 Requirements: What You'll Need

This process has three main parts: installing R, installing RStudio, and setting up Git and a GitHub account. We'll cover steps for both macOS and Windows. Most of you have R and RStudio already, so you can skip those parts and move to Part 3.

3.1 Part 1: Install R, the Engine

R is the programming language that powers everything.

3.1.1 For macOS

1. Go to the [CRAN macOS page](#).

2. Download the .pkg file for the latest R version. It will be named something like R-4.x.x.pkg. Be sure to download the correct version for your Mac's processor (Apple Silicon M1/M2/M3 or Intel).
3. Open the downloaded file and follow the installation prompts. It's a standard macOS installation.

3.1.2 For Windows

1. Go to the [CRAN Windows page](#).
2. Click the big link at the top that says Download R-4.x.x for Windows.
3. Run the downloaded .exe file. It's safe to accept all the default settings during installation.

3.2 Part 2: Install RStudio, the Cockpit

RStudio is the integrated development environment (IDE) we'll use. It gives us a great interface for R and Quarto.

1. Go to the [RStudio Desktop download page](#).
2. The website should automatically detect your operating system and recommend the correct installer.
3. Download and run the installer (.dmg for Mac, .exe for Windows) and follow the prompts.

3.3 Part 3: Get Git and a GitHub Account

We need Git to track our file changes and GitHub to host our website online for free.

3.3.1 Install Git

- **macOS:** Git usually comes pre-installed. You can check by opening the Terminal app and typing `git --version`. If it's not there, it will likely prompt you to install the Xcode Command Line Tools. Agree to the installation. I'm using git version 2.39.5 (Apple Git-154).
- **Windows:** You will likely need to install Git separately. Download it from [git-scm.com](#). Run the installer and you can safely accept all the default settings.

3.3.2 Create a GitHub Account and Install GitHub Desktop

If you don't already have one, go to [github.com](#) and sign up. Please consider a proper username because we are going to use it to host your website. Something like `your-username.github.io` is ideal. For example, my name website is `rrmaximiliano.github.io`.

So, your username url to Github would look like this: `github.com/your-username`; your repository to host your website would look like this: `github.com/your-username/your-username.github.io`; and your website would look like this: `your-username.github.io`.

To install GitHub Desktop, go to [desktop.github.com](#) and download the installer for your operating system. Follow the installation prompts. You will be asked to log in with your GitHub account. We will come back to Github Desktop when we are ready to deploy our website.

3.4 Final Check

Once everything is installed, open RStudio. It should automatically find your R installation. To make sure Quarto is ready, go to Tools > Global Options > R Markdown. You should see a section for Quarto, indicating it's ready to go. Recent versions of RStudio bundle Quarto, so you shouldn't need to install it separately.

Understanding Version Control: Git, GitHub, and GitHub Desktop. Before we dive into using Git and GitHub, let's clarify what these tools are and how they fit into the process of version control.

What is Version Control?

Version control is a system that records changes to files over time so that you can recall specific versions later. It is essential for collaborative work and for keeping track of your project's history. Think of it as a series of **snapshots** of your project at different points in time.

Example: You make changes to your website, save your progress, and can always go back to a previous version if something breaks.

1: Git – The Local Version Control System

Git is a tool that runs on your computer. It tracks changes to your files and lets you save “snapshots” (called *commits*) of your project. You can move back and forth between these snapshots, compare them, and even work on different versions (branches) at the same time.

Example: You add a new section to your website and commit the change. If you later decide you don't like it, you can revert to the previous commit.

2: GitHub – The Remote Hosting Service

GitHub is a website that hosts your Git repositories online. It acts as a backup and makes it easy to collaborate with others. You “push” your local commits to GitHub, and others can “pull” them to their computers.

Example: After making changes and committing them locally with Git, you push your project to GitHub so it's available online and can be shared with collaborators.

3: GitHub Desktop is the friendly interface

GitHub Desktop is an application that makes it easier to use Git and GitHub without typing commands in the terminal. It lets you see your changes, make commits, and push/pull to GitHub with a few clicks.

Instead of typing `git commit` and `git push` in the terminal, you can click buttons in GitHub Desktop to save and upload your changes.

Tool	What it does	Where it runs
Git	Tracks changes to files (version control)	Your computer
GitHub	Hosts your code online (backup/collaboration)	The web
GitHub Desktop	User-friendly interface for Git/GitHub	Your computer

Snapshots Analogy

Imagine writing a book:

- **Git:** You save a copy of your manuscript every time you finish a chapter.
- **GitHub:** You upload those copies to a cloud drive so you can access them anywhere and share with others.
- **GitHub Desktop:** You use a simple app to manage your copies and uploads, instead of using command-line tools.

4 Let's Build! Creating Your Quarto Website

Time for the fun part. We'll create a new website project in RStudio.

4.1 Step 1: Create a New Project

1. Open RStudio.
2. Go to File > New Project ...
3. In the dialog box, select New Directory.
4. Select Quarto Website.
5. Give your project a directory name. This will be the folder on your computer where all your website files live. Something like `my-academic-website` is a good start. For example, I use `rrmaximiliano.github.io` as my project folder as well.

6. Click Create Project.

RStudio will work its *dark magic* and create a new folder with some files in it. You'll see an `index.qmd` file, a `_quarto.yml` file, and maybe an `about.qmd` and a `.gitignore` file.

4.2 Step 2: The Key Files - Examples & Explanation

Let's look at the two most important files: `_quarto.yml` (the brain) and `index.qmd` (the face).

4.2.1 The Brain: `_quarto.yml`

This file controls the entire site's settings. Indentation is very important here, so be careful! By default, your `_quarto.yml` file will look something like this:

```
1 project:
2   type: website
3   title: "test-website"
4
5 website:
6   title: "test-website"
7   navbar:
8     left:
9     - href: index.qmd
10       text: Home
11     - about.qmd
12
13 format:
14   html:
15     theme: cosmo
16     css: styles.css
17     toc: true
```

Listing 1: Example `_quarto.yml` file

The first modification is the output directory. By default, Quarto will render your website to a folder called `_site`. However, if you want to host it on GitHub Pages, you need to change this to `docs`. There is another modification in the following code snippet as well. The first one is the navbar. The navbar serves as a way to navigate your website and can include links to different pages. The common practice is to have a Home page, an About Me page, and a Research page.

You can also have it as a standalone website, meaning that you don't need to have a Home page. In that case, you can remove the navbar section and just have the `index.qmd` file as your main page. I will give you examples of that modification.

```
1 # This section defines the project type and output directory.
2 project:
3   type: website
4   title: "test-website"
5   output-dir: docs # We use 'docs' for GitHub Pages hosting.
6
7 # This section controls the main website settings.
8 website:
9   title: "It's better if you use your name here." # The title that appears in the browser tab.
10  navbar:
11    left:
12    - href: index.qmd
13      text: Home
14    - href: research.qmd # A new page we might create for our research.
15      text: Research
16    - href: about.qmd
17      text: About Me
18  # You can find more themes at https://quarto.org/docs/output-formats/html-themes.html
19  theme: cosmo
20
21 # This section defines the format-specific options, here for HTML output.
22 format:
23   html:
24     css: styles.css # Link to a custom stylesheet.
25     toc: true
```

Listing 2: Modified `_quarto.yml` file

4.2.2 The Face: index.qmd

This is your homepage. It uses a mix of YAML at the top (the part between `---`) for page-specific settings and Markdown for the content.

```
1 ---
2 title: "Welcome"
3 ---
4
5 # PhD Student's Name
6
7 ![A profile picture of PhD Student's Name](profile.jpg)
8 <!-- Make sure you have a profile.jpg in your project folder! -->
9
10 I am a PhD candidate at the University of Awesome, where I study the migratory patterns of rubber ducks.
    My research interests include computational hydrology, data visualization, and bath-time dynamics.
11
12 This website is where I share my latest research, publications, and teaching materials. Feel free to
    look around!
13
14 ## Recent News
15 - **July 2025:** Started my dissertation research on advanced splashing techniques.
16 - **May 2025:** Presented my work at the International Conference of Bath Toys.
17
18 ## Contact
19 You can reach me at \texttt{phd.student@university.edu}.
```

Listing 3: Example index.qmd file

Notice that we are using markdown syntax here. You can use `#` for headings, `*` for bullet points, and `![Alt text](image.jpg)` for images. The YAML at the top sets the page title. For now, I encourage you to continue using markdown syntax, but you can also use HTML if you want to get fancy.

For more references about markdown syntax, you can check out the [Quarto Markdown Basics](#) documentation.

4.3 Step 3: Render and Preview

In RStudio, you should see a **Render** button at the top of the script editor when you have an `index.qmd` file open. Click it!

RStudio will build your website and show you a preview in the Viewer pane or a new window. It's a live preview, so as you make changes to your files and save them, the preview will update automatically. Pretty cool, right?

5 Hosting on GitHub Pages

Okay, you've got a website on your computer. Now let's get it online so others can see it.

5.1 Step 1: Connect Your Project with GitHub Desktop

Instead of using the command line, we'll use the much friendlier GitHub Desktop app to connect your local project folder to the GitHub repository you just created.

1. Open the GitHub Desktop application.
2. Go to `File > Add Local Repository ...`.
3. Click **Choose...** and navigate to the folder where you created your RStudio Quarto project (e.g., `my-academic-website`). Select that folder.
4. GitHub Desktop will recognize it's a Git repository but that it's not on GitHub yet. You will see a button that says **Publish repository**. Click it.
5. In the window that appears, make sure the "Name" matches your GitHub repository name: `(your-username.github.io)`.

6. Uncheck the box that says “Keep this code private.” Your website repository must be public.
7. Click the **Publish Repository** button at the bottom. This will upload your project files for the first time and connect the local folder to the remote repository on GitHub.

5.2 Step 1.5: Configure GitHub Pages

We’re almost there! We just need to tell GitHub where to find the website files.

1. Go to your repository on the GitHub website.
2. Click on the **Settings** tab.
3. On the left-hand menu, click on **Pages**.
4. Under “Build and deployment,” for the “Source,” select **Deploy from a branch**.
5. For the branch, select **main** (or **master**) and the folder **/docs**.
6. Click **Save**.

After a minute or two, your website should be live at <https://your-username.github.io>. Congratulations! You now have a live, public, academic website.

5.3 Step 2: Push and Update Your Website to GitHub

Now for the easy part. Whenever you want to update your live website, you’ll follow this simple workflow.

1. **Render your site in RStudio.** Make sure your `_quarto.yml` file is set to output to the `docs` folder. Click the **Render** button in RStudio to build the latest version of your website files inside the `/docs` folder.
2. **Switch to GitHub Desktop.** The application will automatically detect all the new and changed files in your project. You will see them listed on the left.
3. **Commit your changes.** At the bottom left of the GitHub Desktop window, you’ll see a “Summary” field. Write a short description of your changes (e.g., “Update homepage” or “Add research page”).
4. Click the blue **Commit to main** button. This saves a snapshot of your changes locally.
5. **Push your changes to GitHub.** After committing, the main button at the top of the window will change to **Push origin**. Click it. This sends your committed changes up to GitHub, which will trigger the update to your live website.
6. **Check your website.** After a minute or two, go to <https://your-username.github.io> in your web browser. You should see your updated website live!
7. Add loop all the steps above to your workflow. Whenever you make changes to your website files, just render in RStudio, commit in GitHub Desktop, and push to GitHub.

6 Troubleshooting: Common Issues with GitHub Pages, Jekyll, and Quarto

Even with careful setup, you may encounter issues when deploying your Quarto website to GitHub Pages. Here are some common problems and their solutions:

6.1 Website Not Updating or Not Showing Up

- GitHub Pages can take a few minutes to build and deploy your site after you push changes.
- Make sure GitHub Pages is set to deploy from the **main** branch and the `/docs` folder.
- Sometimes your browser caches the old version. Try a hard refresh (Shift+Reload or Ctrl+F5).

6.2 Jekyll Processing Issues (.nojekyll File)

GitHub Pages uses Jekyll by default, which ignores files and folders that start with an underscore (like `_quarto.yml` or `_site`). To prevent this:

- Quarto automatically creates a `.nojekyll` file in your output directory (`docs` or `_site`). This tells GitHub Pages **not** to use Jekyll.
- If your site is missing files or not rendering correctly, check that the `.nojekyll` file exists in your output directory and is committed to your repository.
- If it's missing, re-render your site and make sure to commit and push the `.nojekyll` file.

6.3 Broken Links or Missing Images

- **Check File Paths:** All file and image paths are case-sensitive on GitHub Pages. Make sure the filenames and paths match exactly.
- **Relative Paths:** Use relative paths (e.g., `images/photo.jpg`) rather than absolute paths.
- **Commit All Files:** Make sure you have added and committed all images and resources to your repository.

6.4 Changes Not Appearing

- **Did You Render?** Always click Render in RStudio before committing and pushing changes. Only the files in the output directory (`docs` or `_site`) are served by GitHub Pages.
- **Case Sensitivity:** Again, GitHub Pages is case-sensitive. `About.html` and `about.html` are different files.
- Try to always render from the main `index.qmd` file. If you render from a subpage, it may not update the main page correctly.

I use the following configuration in my `_quarto.yml` file to ensure that the main page is always rendered:

```
output-dir: docs
render:
  - "*.qmd"
```

This configuration ensures that all `.qmd` files are rendered, including the main `index.qmd` file.

- **Did You Commit?** After rendering, make sure you commit the changes in the `Git` tab of RStudio.
- **Did You Push?** After rendering, commit and push your changes to GitHub.

6.5 404 Errors on Subpages

- **Check Navigation Links:** Make sure your `_quarto.yml` navbar links match the actual filenames (e.g., `about.qmd` renders to `about.html`).
- **Case Sensitivity:** Again, again, again, GitHub Pages is case-sensitive. `About.html` and `about.html` are different files.

6.6 YAML Formatting Errors

- YAML is very sensitive to indentation. Use spaces, not tabs, and keep indentation consistent.
- Make sure you use colons correctly and quote strings with special characters.
- If rendering fails, check the RStudio console for YAML parsing errors.

6.7 Quarto Version Issues

- If you see errors about unknown options or rendering failures, make sure you are using the latest version of Quarto. Update via `quarto update` in the terminal or download from quarto.org.

6.8 More Help

- Quarto Documentation: [Publishing to GitHub Pages](#)
- GitHub Pages Documentation: [GitHub Pages Help](#)
- Happy Git with R: [Comprehensive guide for using Git and GitHub with RStudio](#)
- Quarto Community: [Quarto Community Resources](#)

If you get stuck, just ping me! :) Most issues are due to small typos or missed steps.